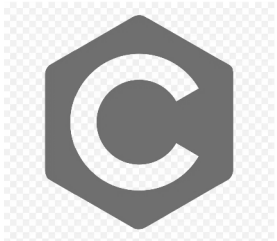
COMPE496 // INTRO TO ALGORITHMS

C++ Programming Language

Concepts and Introduction



AGENDA

01 Foundations

History, Syntax Differences, and Basic Types

02 Object-Oriented Core

Classes, Objects, Inheritance, and Polymorphism

03 Memory Management

Pointers, References, and Resource Management

04 Generic Programming

Templates and the Standard Template Library (STL)

05 Modern C++

Smart Pointers, Auto, and Best Practices

Introduction to C++

Insight: C++ was designed as a superset of C, adding Object-Oriented features without sacrificing low-level performance.

ORIGINS

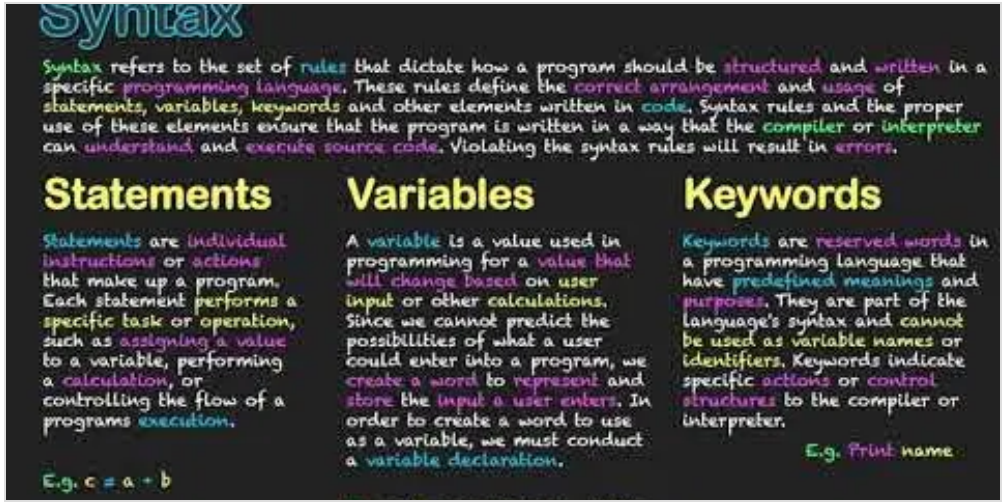
Developed by **Bjarne Stroustrup** at Bell Labs in 1979. Originally named `"C with Classes"`, it aimed to add Simula's organizational features to C's raw power.

THE SUPERSET PHILOSOPHY

C++ is (mostly) a superset of C. This means virtually all valid C code is valid C++. It maintains direct memory manipulation and compilation to machine code.

CORE GOAL

To provide **Zero-Overhead Abstraction**. You don't pay for features you don't use, and high-level abstractions should be as efficient as hand-written code.



→ **Object-Oriented**
Classes, Inheritance, Polymorphism.

→ **Strongly Typed**
Stricter type checking than C to catch errors early.

→ **Standard Template Library (STL)**
Rich collection of algorithms and data structures.

C vs C++ Syntax

Insight: C++ modernizes C with stronger type safety, native boolean support, and namespaces to organize code and prevent collisions.

Namespaces

C++ uses `namespace` (like `std::`) to group identifiers, solving the global name pollution problem of C.

Boolean Type

C++ has a native `bool` type with `true` and `false` keywords. C traditionally used integers (0/1).

I/O Streams

Type-safe streams (`cin`, `cout`) replace the format-specifier-heavy `printf` and `scanf`.

TRADITIONAL C — MANUAL & RAW

```c
#include <stdio.h>
int main() { int success = 1; // 0 or 1 if (success) { // %s, %d required
printf("Status: %d\n", success); } return 0; }
```

MODERN C++ — TYPE-SAFE & EXPRESSIVE

```cpp
#include <iostream> using namespace std; int main() { bool success = true; //
Native bool if (success) { // No format specifiers needed cout << "Status: "
<< success << endl; } return 0; }
```

Object-Oriented Programming

Insight: The three pillars of OOP—Encapsulation, Inheritance, and Polymorphism—enable the creation of modular, reusable, and scalable software systems.

Encapsulation

Bundling data and methods together. Hiding internal state to protect integrity.

```
class Account { private: double
balance; public: void
deposit(double amt) { if (amt > 0)
balance += amt; } };
```

Inheritance

Creating new classes from existing ones. The "Is-A" relationship promotes code reuse.

```
class Animal { ... }; // Dog
inherits from Animal class Dog :
public Animal { public: void bark()
{ ... } };
```

Polymorphism

One interface, many forms. Objects behave differently based on their specific type.

```
virtual void draw() = 0; //
Different implementations
Circle::draw() { ... }
Square::draw() { ... }
```

Classes and Objects

Insight: A Class is a blueprint defining structure and behavior; an Object is a concrete instance of that blueprint in memory.

The Blueprint

Classes encapsulate data (attributes) and functions (methods) into a single unit.

Access Specifiers

`public:` Accessible from anywhere.

`private:` Accessible only within the class.

`protected:` Accessible by derived classes.

Syntax Trap

Don't forget the semicolon ; at the end of the class definition!

```
class Rectangle { private: int width, height; public: void setValues(int w, int h)
{ width = w; height = h; } int area() { return width * height; } }; // ⟵
Semicolon is mandatory int main() { Rectangle rect; // Create Object
rect.setValues(3, 4); // Access Method cout << "Area: " << rect.area(); return 0; }
```

Constructors and Destructors

Insight: Constructors birth objects into a valid state; Destructors ensure a clean exit, preventing resource leaks (RAII).

Constructor (Init)

- Same name as the class.
- No return type (not even void).
- Called automatically upon creation.
- Can be overloaded (multiple versions).

```
class Box { public: // Default Constructor Box() { cout << "Box Created"; } // Parameterized Constructor Box(int size) { ... } };
```

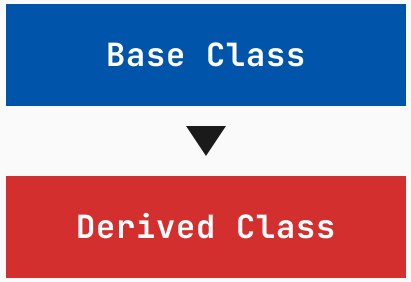
Destructor (Cleanup)

- Same name with ~ prefix.
- No parameters, no return type.
- Called automatically on scope exit.
- Cannot be overloaded (only one).

```
class Box { public: // Destructor ~Box() { cout << "Box Destroyed"; // Release resources here // e.g., delete ptr; } };
```

Inheritance

Insight: Inheritance allows new classes to adopt and extend the behavior of existing ones, creating an "Is-A" relationship.



Derived inherits from Base

Public Inheritance Rules

BASE MEMBER	IN DERIVED CLASS
public	public
protected	protected
private	*Inaccessible*

```
class Animal { public: void eat() { cout << "Eating..." << endl; } }; // Dog inherits from Animal class Dog : public Animal { public: void bark() { cout << "Woof!" << endl; } }; int main() { Dog myDog; // Inherited method myDog.eat(); // Output: Eating... // Own method myDog.bark(); // Output: Woof! return 0; }
```

Polymorphism

Insight: Polymorphism allows a single interface to represent different underlying forms. It comes in two flavors: Static (Compile-time) and Dynamic (Runtime).

Overloading

COMPILE-TIME

Multiple functions with the same name but different parameter lists. The compiler decides which one to call based on the arguments.

```
class Printer { public: void print(int i) { cout << "Int:
" << i << endl; } void print(double f) { cout << "Float: "
<< f << endl; } }; // Usage Printer p; p.print(5); //
Calls int version p.print(3.14); // Calls double version
```

Overriding

RUNTIME

Derived classes provide specific implementations of `virtual` functions. The call is resolved at runtime based on the object type.

```
class Base { public: virtual void show() { cout << "Base";
} }; class Derived : public Base { public: void show()
override { cout << "Derived"; } }; // Usage Base* ptr =
new Derived(); ptr→show(); // Output: Derived
```

References vs Pointers

Insight: References act as strict aliases for existing variables, offering a safer and cleaner syntax compared to the raw memory power of pointers.

Pointers `*`

- **Reassignable:** Can point to different objects over time.
- **Nullable:** Can be set to `nullptr` (point to nothing).
- **Explicit Access:** Requires dereferencing (*) to access value.
- **Memory Address:** Stores the actual memory address.

```
int a = 10; int b = 20; int* ptr = &a; // ptr holds address of a
*ptr = 30; // a becomes 30 ptr = &b; // Now points to b ptr =
nullptr; // Valid
```

References `&`

- **Fixed Binding:** Cannot be reseated to refer to another object.
- **Never Null:** Must be initialized when declared.
- **Implicit Access:** Used exactly like the original variable.
- **Alias:** Just another name for the same memory.

```
int a = 10; int b = 20; int& ref = a; // ref is alias for a ref =
30; // a becomes 30 ref = b; // Assigns value of b to a! // ref
still refers to a
```

Templates

Insight: Templates enable generic programming, allowing you to write a single function or class that works with any data type.

Generic Programming

Instead of writing separate functions for `int`, `float`, and `double`, write one template. The compiler generates the specific code needed.

The Syntax

Use the keyword `template` followed by generic parameters inside angle brackets `<typename T>`.

Benefits

- Code Reusability
- Type Safety (checked at compile time)
- Performance (no runtime overhead)

FUNCTION TEMPLATE

```
template <typename T> T add(T a, T b) { return a + b; } int main() { add<int>(5, 10); // 15 add<double>(2.5, 3.1); // 5.6 }
```

CLASS TEMPLATE

```
template <class T> class Box { T value; public: Box(T v) : value(v) {} T getValue() { return value; } }; int main() { Box<int> intBox(100); Box<string> strBox("Hello"); }
```

Standard Template Library (STL)

Insight: The STL provides a rich set of generic classes and functions, allowing developers to write efficient, reusable code without reinventing data structures.

I/O Streams

Insight: C++ abstracts input and output as streams of bytes, providing a uniform, type-safe interface for both console and file operations.

STANDARD OBJECTS

`cin`	Standard Input (Keyboard)
`cout`	Standard Output (Screen)
`cerr`	Standard Error (Unbuffered)

OPERATORS

<< **Insertion Operator**
Sends data TO an output stream.

>> **Extraction Operator**
Gets data FROM an input stream.

CONSOLE I/O (<IOSTREAM>)

```
int age; // Output (Insertion) cout << "Enter your age: "; // Input (Extraction) cin >> age; cout << "You are " << age << " years old." << endl;
```

FILE I/O (<FSTREAM>)

```
#include <fstream> // Writing to a file ofstream outFile("data.txt"); if (outFile.is_open()) { outFile << "Saving data: " << 42; outFile.close(); } // Reading works similarly with ifstream
```

Modern C++ Features

Insight: Modern C++ (C++11+) prioritizes safety and conciseness. Features like smart pointers and type inference eliminate common sources of bugs and verbosity.

Smart Pointers

Wrappers that manage memory automatically (RAII). They eliminate manual `delete` calls and prevent memory leaks.

```
#include <memory> void func() { // Unique ownership auto ptr = std::make_unique<int>(10); // No delete needed! // Memory freed when ptr goes out of scope }
```

Type Inference (auto)

Lets the compiler deduce the variable type from its initializer. Reduces verbosity, especially with complex template types.

```
// Compiler knows i is int auto i = 42; // Very useful for iterators std::vector<int> vec; for (auto it = vec.begin(); it != vec.end(); ++it) { // ... }
```

Lambda Expressions

Anonymous function objects defined inline. Perfect for passing short snippets of logic to algorithms.

```
std::vector<int> v = {1, 2, 3}; // [capture](params){body} std::for_each(v.begin(), v.end(), [](int n) { std::cout << n * n << " "; }); // Output: 1 4 9
```

Summary

Insight: C++ balances high-level abstraction with low-level hardware control.

OBJECT-ORIENTED

- **Classes & Objects:** The blueprint and the instance.
- **Encapsulation:** Protecting data integrity.
- **Polymorphism:** Flexible interfaces via virtual functions.

SYSTEMS & MEMORY

- **Pointers:** Direct memory access and manipulation.
- **RAII:** Resource management tied to object lifetime.
- **Performance:** Zero-overhead abstractions.

MODERN & GENERIC

- **Templates:** Write once, use with any type.
- **STL:** Battle-tested containers and algorithms.
- **Smart Pointers:** Automated memory management.

"C++ makes it harder to shoot yourself in the foot, but when you do, you blow off your whole leg." — Bjarne Stroustrup